

Empleados

Pepe Calderón, Fernando Sola, Daniel Ayala, Ana Belén Sánchez, Inma Hernández, Margarita Cruz, Carlos Arévalo, David Ruiz



Escuela Técnica Superior de
Ingeniería Informática

1. Requisitos de información (RI)	2
1.1. RI-01: Departamentos	2
1.2. RI-02: Empleados	2
2. Reglas de negocio (RN)	3
2.1. RN-01: Departamentos	3
2.2. RN-02: Empleados	3
3. Diagrama de clases	4
4. Álgebra relacional	6
5. Script SQL para crear la base de datos	10
6. Script SQL para la carga inicial de datos	12
7. Consultas	14
8. SQL avanzado	19
8.1. Procedimientos almacenados	19
8.2. Funciones	22
8.3. Cursores	23
8.4. Triggers (disparadores)	24
9. Pruebas SQL	27

Requisitos Se pretende realizar un pequeño sistema de información para gestionar los empleados de los departamentos de una empresa. Cada empleado puede pertenecer a cero o un departamento y puede tener un jefe. Cada departamento pertenece a una localidad. Cada empleado tiene un salario y una comisión; también se almacenan las fechas de inicio y, si existe, de finalización de su contrato.

Requisitos de información (RI)

1.1. RI-01: Departamentos

- **Como:** Profesor de la asignatura
- **Quiero:** Poder almacenar la siguiente información de los departamentos: nombre del departamento(obligatorio) y localidad(opcional)
- **Para:** Que el alumno practique con un sistema de información sencillo

1.2. RI-02: Empleados

- **Como:** Profesor de la asignatura
- **Quiero:** Poder almacenar la siguiente información de los empleados: nombre (obligatorio y único), salario, comisión y fecha de inicio (obligatorios; si no se indica esta última, se asigna la fecha actual), fecha de fin (opcional), jefe (opcional) y departamento al que pertenece (opcional)
- **Para:** Que el alumno practique con un sistema de información sencillo

Reglas de negocio (RN)

2.1. RN-01: Departamentos

- **Como:** Profesor de la asignatura
- **Quiero:** Un departamento no puede tener más de cinco empleados y la combinación nombre y localidad informada no puede repetirse. Si la localidad es NULL, pueden existir varios departamentos con el mismo nombre porque la localidad es desconocida.
- **Para:** Que el alumno practique con restricciones simples

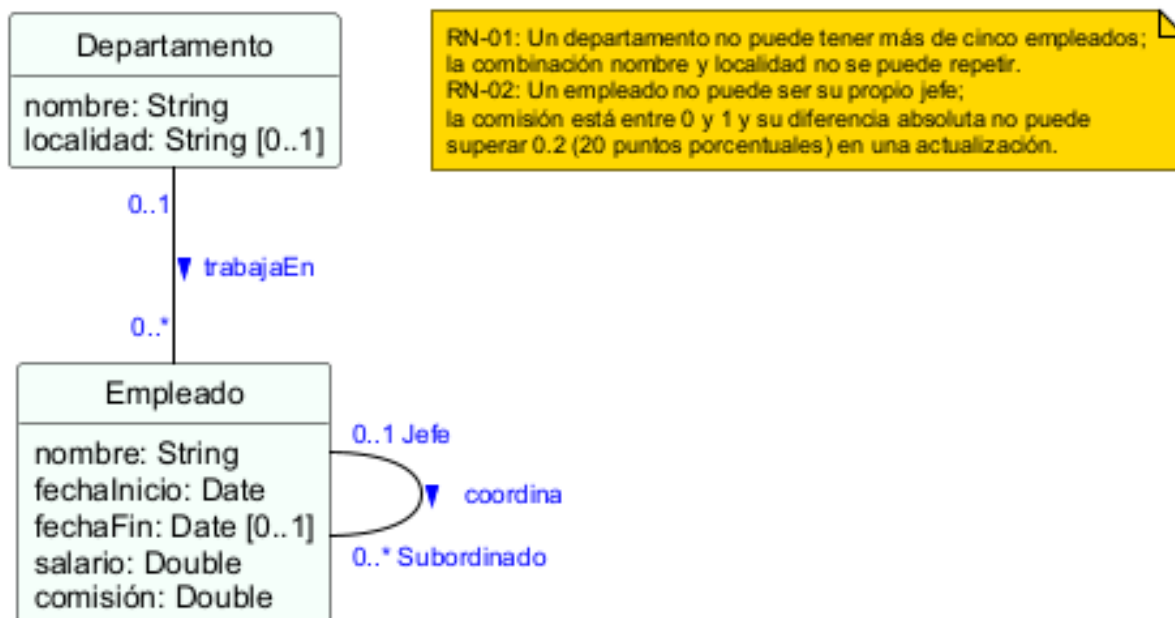
2.2. RN-02: Empleados

- **Como:** Profesor de la asignatura
- **Quiero:** Un empleado no puede ser jefe de si mismo; la comisión es un porcentaje del salario, no puede ser negativa y no puede variar en más de 20 puntos porcentuales de golpe
- **Para:** Que el alumno practique con restricciones simples

Modelo Conceptual

Diagrama de clases

Empleados: diagrama de clases



Modelo Relacional

```

Departamentos = { departamentoId, nombre, localidad }
  PK(departamentoId)
  AK(nombre, localidad)
Empleados = { empleadoId, departamentoId, jefeId, nombre, salario, fechaInicio, fechaFin,
comision }
  
```

3. Diagrama de clases

```
PK(empleadoId)
FK(departamentoId) / Departamentos
FK(jefeId) / Empleados
AK(nombre)
Departamentos = {
  (d1, 'Arte', 'Cádiz'),
  (d2, 'Historia', null),
  (d3, 'Informática', 'Sevilla')
}
Empleados = {
  (e1, d1, NULL, 'Pedro', 2300.00, '2017-09-15', NULL, 0.2),
  (e2, d1, NULL, 'Jose', 2500.00, '2018-08-15', NULL, 0.5),
  (e3, d2, NULL, 'Lola', 2300.00, '2018-08-15', NULL, 0.3),
  (e4, d1, e1, 'Luis', 1300.00, '2018-08-15', '2018-11-15', 0),
  (e5, d1, e1, 'Ana', 1300.00, '2018-08-15', '2018-11-15', 0),
  (e6, d1, e1, 'Daniel', 1500.00, '2020-09-15', NULL, 0.2),
  (e7, NULL, NULL, 'Eva', 1200.00, '2021-05-01', NULL, 0.1)
}
```

Álgebra relacional

- Renombrado de relaciones para acortar las expresiones en álgebra relacional:

$$\rho_{D(dId, dn, l)}(Departamentos)$$

$$\rho_{E(eId, dId, jId, en, s, fI, fF, c)}(Empleados)$$

- Empleados con sueldo < 2000:

$$\sigma_{s < 2000}(E)$$

```
EmpleadosSueldoBajo = { eId, dId, jId, en, s, fI, fF, c }
```

```
EmpleadosSueldoBajo = {
  (e4, d1, e1, 'Luis', 1300.00, '2018-08-15', '2018-11-15', 0),
  (e5, d1, e1, 'Ana', 1300.00, '2018-08-15', '2018-11-15', 0),
  (e6, d1, e1, 'Daniel', 1500.00, '2020-09-15', NULL, 0.2),
  (e7, NULL, NULL, 'Eva', 1200.00, '2021-05-01', NULL, 0.1)
}
```

- Fechas de alta y baja:

$$\Pi_{fI, fF}(E)$$

```
FechasContratos = { fI, fF }
```

```
FechasContratos = {
  ('2017-09-15', NULL),
  ('2018-08-15', NULL),
}
```

4. Álgebra relacional

```
('2018-08-15', '2018-11-15'),  
( '2020-09-15', NULL),  
( '2021-05-01', NULL)  
}
```

- Sueldo entre 2000 y 3000:

$$\sigma_{2000 < s < 3000}(E)$$

```
EmpleadosSueldoMedio = { eId, dId, jId, en, s, fI, fF, c }
```

```
EmpleadosSueldoMedio = {  
  (e1, d1, NULL, 'Pedro', 2300.00, '2017-09-15', NULL, 0.2),  
  (e2, d1, NULL, 'Jose', 2500.00, '2018-08-15', NULL, 0.5),  
  (e3, d2, NULL, 'Lola', 2300.00, '2018-08-15', NULL, 0.3)  
}
```

- Producto cartesiano:

$$E \times D$$

```
ProductoEmpleadosDepartamentos = { eId, E.dId, jId, en, s, fI, fF, c, D.dId, dn, l }
```

```
ProductoEmpleadosDepartamentos = {  
  (e1, d1, NULL, 'Pedro', 2300.00, '2017-09-15', NULL, 0.2, d1, 'Arte', 'Cádiz'),  
  (e1, d1, NULL, 'Pedro', 2300.00, '2017-09-15', NULL, 0.2, d2, 'Historia', NULL),  
  (e1, d1, NULL, 'Pedro', 2300.00, '2017-09-15', NULL, 0.2, d3, 'Informática', 'Sevilla'),  
  (e2, d1, NULL, 'Jose', 2500.00, '2018-08-15', NULL, 0.5, d1, 'Arte', 'Cádiz'),  
  (e2, d1, NULL, 'Jose', 2500.00, '2018-08-15', NULL, 0.5, d2, 'Historia', NULL),  
  (e2, d1, NULL, 'Jose', 2500.00, '2018-08-15', NULL, 0.5, d3, 'Informática', 'Sevilla'),  
  (e3, d2, NULL, 'Lola', 2300.00, '2018-08-15', NULL, 0.3, d1, 'Arte', 'Cádiz'),  
  (e3, d2, NULL, 'Lola', 2300.00, '2018-08-15', NULL, 0.3, d2, 'Historia', NULL),  
  (e3, d2, NULL, 'Lola', 2300.00, '2018-08-15', NULL, 0.3, d3, 'Informática', 'Sevilla'),  
  (e4, d1, e1, 'Luis', 1300.00, '2018-08-15', '2018-11-15', 0, d1, 'Arte', 'Cádiz'),  
  (e4, d1, e1, 'Luis', 1300.00, '2018-08-15', '2018-11-15', 0, d2, 'Historia', NULL),  
  (e4, d1, e1, 'Luis', 1300.00, '2018-08-15', '2018-11-15', 0, d3, 'Informática', 'Sevilla'),  
  (e5, d1, e1, 'Ana', 1300.00, '2018-08-15', '2018-11-15', 0, d1, 'Arte', 'Cádiz'),  
  (e5, d1, e1, 'Ana', 1300.00, '2018-08-15', '2018-11-15', 0, d2, 'Historia', NULL),  
  (e5, d1, e1, 'Ana', 1300.00, '2018-08-15', '2018-11-15', 0, d3, 'Informática', 'Sevilla'),  
  (e6, d1, e1, 'Daniel', 1500.00, '2020-09-15', NULL, 0.2, d1, 'Arte', 'Cádiz'),  
  (e6, d1, e1, 'Daniel', 1500.00, '2020-09-15', NULL, 0.2, d2, 'Historia', NULL),  
  (e6, d1, e1, 'Daniel', 1500.00, '2020-09-15', NULL, 0.2, d3, 'Informática', 'Sevilla'),  
  (e7, NULL, NULL, 'Eva', 1200.00, '2021-05-01', NULL, 0.1, d1, 'Arte', 'Cádiz'),  
  (e7, NULL, NULL, 'Eva', 1200.00, '2021-05-01', NULL, 0.1, d2, 'Historia', NULL),  
  (e7, NULL, NULL, 'Eva', 1200.00, '2021-05-01', NULL, 0.1, d3, 'Informática', 'Sevilla')  
}
```

- Join Empleados-Departamentos:

$$E \bowtie D$$

```
EmpleadosDepartamentos = { eId, dId, jId, en, s, fI, fF, c, dn, l }
```

```
EmpleadosDepartamentos = {  
  (e1, d1, NULL, 'Pedro', 2300.00, '2017-09-15', NULL, 0.2, 'Arte', 'Cádiz'),  
  (e2, d1, NULL, 'Jose', 2500.00, '2018-08-15', NULL, 0.5, 'Arte', 'Cádiz'),  
  (e3, d2, NULL, 'Lola', 2300.00, '2018-08-15', NULL, 0.3, 'Historia', NULL),  
}
```

```
(e4, d1, e1, 'Luis', 1300.00, '2018-08-15', '2018-11-15', 0, 'Arte', 'Cádiz'),
(e5, d1, e1, 'Ana', 1300.00, '2018-08-15', '2018-11-15', 0, 'Arte', 'Cádiz'),
(e6, d1, e1, 'Daniel', 1500.00, '2020-09-15', NULL, 0.2, 'Arte', 'Cádiz')
}
```

- Departamentos con empleados:

$$\Pi_{dId}(E \bowtie D)$$

```
DepartamentosConEmpleados = { dId }

DepartamentosConEmpleados = {
  (d1),
  (d2)
}
```

- Departamentos sin empleados:

$$\Pi_{dId}(D) - \Pi_{dId}(E \bowtie D)$$

```
DepartamentosSinEmpleados = { dId }

DepartamentosSinEmpleados = {
  (d3)
}
```

- Estadísticas globales de salario:

$$\rho_{total}(\text{COUNT}(*)), \rho_{minSalario}(\text{MIN}(s)), \rho_{maxSalario}(\text{MAX}(s)), \rho_{mediaSalario}(\text{AVG}(s)), \rho_{sumaSalarios}(\text{SUM}(s))_{\gamma}(E)$$

```
EstadisticasSalario = { total, minSalario, maxSalario, mediaSalario, sumaSalarios }

EstadisticasSalario = {
  (7, 1200.00, 2500.00, 1771.43, 12400.00)
}
```

- Estadísticas de salario por departamento:

$$dId, \rho_{total}(\text{COUNT}(*)), \rho_{minSalario}(\text{MIN}(s)), \rho_{maxSalario}(\text{MAX}(s)), \rho_{mediaSalario}(\text{AVG}(s)), \rho_{sumaSalarios}(\text{SUM}(s))_{\gamma} dId(E \bowtie D)$$

```
EstadisticasSalarioDepartamento = { dId, total, minSalario, maxSalario, mediaSalario,
  sumaSalarios }

EstadisticasSalarioDepartamento = {
  (d1, 5, 1300.00, 2500.00, 1780.00, 8900.00),
  (d2, 1, 2300.00, 2300.00, 2300.00, 2300.00)
}
```

- Estadísticas de salarios por departamento con al menos dos empleados:

$$NumEmpDep \leftarrow \rho_{total}(\text{COUNT}(*))_{\gamma} dId(E \bowtie D)$$

4. Álgebra relacional

```
NumEmpDep = { dId, total }
```

```
NumEmpDep = {  
  (d1, 5),  
  (d2, 1)  
}
```

$$Dep2 \leftarrow \Pi_{dId} \left(\sigma_{total \geq 2} (NumEmpDep) \right)$$

```
Dep2 = { dId }
```

```
Dep2 = {  
  (d1)  
}
```

$$dId, \rho_{total}(\text{COUNT}(*)), \rho_{minSalario}(\text{MIN}(s)), \rho_{maxSalario}(\text{MAX}(s)), \rho_{mediaSalario}(\text{AVG}(s)), \rho_{sumaSalarios}(\text{SUM}(s)) \gamma dId(E \bowtie Dep2)$$

```
EstadisticasSalarioDep2 = { dId, total, minSalario, maxSalario, mediaSalario, sumaSalarios }
```

```
EstadisticasSalarioDep2 = {  
  (d1, 5, 1300.00, 2500.00, 1780.00, 8900.00)  
}
```

Modelo Tecnológico

Script SQL para crear la base de datos

empleados/create_db.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: Script para crear la BD
--

DROP DATABASE if EXISTS EmployeesDB;
CREATE DATABASE EmployeesDB;
USE EmployeesDB;

CREATE OR REPLACE TABLE departments (
  department_id INT NOT NULL AUTO_INCREMENT,
  name_dep VARCHAR(32) NOT NULL,
  city VARCHAR(64),
  CONSTRAINT rn01_department_name_city UNIQUE (name_dep, city),
  PRIMARY KEY(department_id)
);

CREATE OR REPLACE TABLE employees (
  employee_id INT NOT NULL AUTO_INCREMENT ,
  department_id INT,
  boss_id INT,
```

```
name_emp VARCHAR(64) NOT NULL,  
start_date DATE NOT NULL,  
end_date DATE,  
salary DOUBLE NOT NULL DEFAULT 2000,  
fee DOUBLE NOT NULL,  
PRIMARY KEY(employee_id),  
FOREIGN KEY (department_id) REFERENCES departments (department_id)  
    ON DELETE RESTRICT,  
FOREIGN KEY (boss_id) REFERENCES employees (employee_id),  
CONSTRAINT ri02_employee_name UNIQUE(name_emp),  
CONSTRAINT rn02_fee_range CHECK (fee>=0 AND fee<=1),  
CONSTRAINT ck_employee_contract_dates CHECK (start_date < end_date),  
CONSTRAINT valid_salary CHECK (salary > 0)  
);
```

Script SQL para la carga inicial de datos

empleados/populate_db.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: Script para la carga inicial de datos
--

DELIMITER //
CREATE OR REPLACE PROCEDURE p_populate()
BEGIN
    -- desactivar las restricciones de FK para poder borrar, la tabla de
    empleados tiene una FK a si misma.
    SET FOREIGN_KEY_CHECKS = 0;
    DELETE FROM employees;
    DELETE FROM departments;
    SET FOREIGN_KEY_CHECKS = 1;

    INSERT INTO departments (department_id, name_dep, city) VALUES
        (1, 'Arte', 'Cádiz'),
        (2, 'Historia', NULL),
        (3, 'Informática', 'Sevilla')
    ;

    INSERT INTO employees (employee_id, department_id, boss_id, name_emp,
```

```
salary, start_date, end_date, fee) VALUES
  (1, 1, NULL, 'Pedro', 2300.00, '2017-09-15', NULL, 0.2),
  (2, 1, NULL, 'Jose', 2500.00, '2018-08-15', NULL, 0.5),
  (3, 2, NULL, 'Lola', 2300.00, '2018-08-15', NULL, 0.3),
  (4, 1, 1, 'Luis', 1300.00, '2018-08-15', '2018-11-15', 0),
  (5, 1, 1, 'Ana', 1300.00, '2018-08-15', '2018-11-15', 0),
  (6, 1, 1, 'Daniel', 1500.00, '2020-09-15', NULL, 0.2),
  (7, NULL, NULL, 'Eva', 1200.00, '2021-05-01', NULL, 0.1)
;
END;
//
DELIMITER ;

CALL p_populate();
```

Consultas

empleados/queries.sql

```
/* Consultas, para probar las consultas una a una debe ejecutar
Shift+Control+F9 con el cursor en la línea que contiene la consulta */

/* Employees con un sueldo inferior a 2000 euros */
SELECT *
FROM employees
WHERE salary < 2000;

/* Employees con un sueldo inferior a 2000 euros */
SELECT name_emp, salary
FROM employees
WHERE salary < 2000;

/* Fechas de alta y baja de los empleados como una lista */
SELECT ALL start_date, end_date
FROM employees;

/* Fechas de alta y baja de los empleados como un conjunto */
SELECT DISTINCT start_date, end_date
FROM employees;

/* Conjunto de employees con un salary en [2000,3000] */
/* Opción 1 */
SELECT DISTINCT name_emp, salary
FROM employees
WHERE salary >=2000 AND salary <=3000;
```

```

/* Opción 2 */
SELECT DISTINCT name_emp, salary
FROM employees
WHERE salary BETWEEN 2000 AND 3000;

/* Conjunto de employees con salary de 1000, 2500 o 3000 euros */
SELECT DISTINCT name_emp, salary
FROM employees
WHERE salary IN (1000,2500,3000);

/* Lista de employees con una 'o' en la segunda posición de su name_emp
o que son jefes */
SELECT *
FROM employees
WHERE name_emp LIKE '_o%' OR boss_id IS NULL;

/* Lista de employees ordenada por department_id y luego por name_emp */
SELECT *
FROM employees
ORDER BY department_id, name_emp ASC;

/* Producto cartesiano de employees y departments */
SELECT *
FROM employees, departments;

/* Opción 1: */
SELECT name_emp, salary, start_date, name_dep
FROM employees e, departments d
WHERE e.department_id=d.department_id;
/* Opción 2: */
SELECT name_emp, salary, start_date, name_dep
FROM employees NATURAL JOIN departments;

/* Join parciales */
/* LEFT JOIN conserva todos los empleados, incluso si no tienen
departamento */
SELECT e.employee_id, e.name_emp, d.department_id, d.name_dep
FROM employees e
LEFT JOIN departments d
ON e.department_id=d.department_id;

/* RIGHT JOIN conserva igualmente todos los departamentos */
SELECT e.employee_id, e.name_emp, d.department_id, d.name_dep
FROM employees e
RIGHT JOIN departments d
ON e.department_id=d.department_id;

/* Ejemplo de unión de left y right join, devuelve el full join */
SELECT e.employee_id, e.name_emp, d.department_id, d.name_dep
FROM employees e

```

```

LEFT JOIN departments d
ON e.department_id=d.department_id
UNION
SELECT e.employee_id, e.name_emp, d.department_id, d.name_dep
FROM employees e
RIGHT JOIN departments d
ON e.department_id=d.department_id;

/* departments sin employees */
SELECT *
FROM departments d
WHERE NOT EXISTS (
    SELECT * FROM employees e
    WHERE d.department_id=e.department_id
);

/* departments con employees */
SELECT *
FROM departments d
WHERE EXISTS (
    SELECT * FROM employees e
    WHERE d.department_id=e.department_id
);

/* Estadísticas salarys de los employees */
SELECT COUNT(*), MIN(salary), MAX(salary), AVG(salary), SUM(salary)
FROM employees;

/* Estadísticas salarys por departamento */
SELECT e.department_id,
COUNT(*),
AVG(e.salary) avg_salary,
AVG(e.salary * (1+e.fee)) salary_with_fee,
SUM(e.salary) total_salaries
FROM employees e
JOIN departments d ON d.department_id=e.department_id
GROUP BY e.department_id;

/* Estadísticas salarys por departamento con al menos dos empleado*/
SELECT e.department_id,
COUNT(*),
AVG(e.salary) avg_salary,
AVG(e.salary * (1+e.fee)) salary_with_fee,
SUM(e.salary) total_salaries
FROM employees e
JOIN departments d ON d.department_id=e.department_id
GROUP BY e.department_id HAVING COUNT(*)>1;
/* Opción 2: Usando la vista employees_departments */
CREATE OR REPLACE VIEW v_employees_departments AS
SELECT *

```

```

FROM employees NATURAL JOIN departments;

SELECT name_dep,
       COUNT(*),
       AVG(salary) avg_salary,
       AVG(salary * (1+fee)) salary_with_fee,
       SUM(salary) total_salaries
FROM v_employees_departments
GROUP BY department_id HAVING COUNT(*)>1;

/* employees con salary mayor que la media de su departamento*/
SELECT * FROM employees
WHERE salary >
ALL (SELECT AVG(e2.salary)
     FROM employees e2
     JOIN departments d2 ON d2.department_id=e2.department_id
     GROUP BY e2.department_id);

/* Departamento con más employees */
/* Opción 1 */
SELECT e.department_id FROM employees e
JOIN departments d ON d.department_id=e.department_id
GROUP BY e.department_id HAVING COUNT(*)>= ALL
( SELECT COUNT(*)
  FROM employees e2
  JOIN departments d2 ON d2.department_id=e2.department_id
  GROUP BY e2.department_id
);

/* Opción 2 */
SELECT e.department_id FROM employees e
JOIN departments d ON d.department_id=e.department_id
GROUP BY e.department_id HAVING COUNT(*) =
( SELECT MAX(total) FROM
  ( SELECT COUNT(*) AS total
    FROM employees e2
    JOIN departments d2 ON d2.department_id=e2.department_id
    GROUP BY e2.department_id
  ) num_employees
);

/* Vista con las estadísticas de los employees por Departamento */
CREATE OR REPLACE VIEW v_stat_employees AS
SELECT e.department_id,
       COUNT(*) AS num_employees,
       AVG(e.salary) avg_salary,
       AVG(e.salary * (1+e.fee)) salary_with_fee,
       SUM(e.salary) total_salary
FROM employees e

```

```
JOIN departments d ON d.department_id=e.department_id  
GROUP BY e.department_id;
```

```
/* Número de employees que tiene el departamento con más employees */  
SELECT MAX(num_employees)  
FROM v_stat_employees;
```

SQL avanzado

8.1. Procedimientos almacenados

Realice procedimientos para insertar en las tablas Departments y Employees:

empleados/p_insert_department.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: Procedimiento para insertar un nuevo Departamento
--

DELIMITER //
CREATE OR REPLACE PROCEDURE
  p_insert_department(
    p_name_dep VARCHAR(32),
    p_city VARCHAR(64))
BEGIN
  INSERT INTO departments (name_dep, city) VALUES (p_name_dep, p_city);
END//
DELIMITER ;

-- CALL p_populate();
-- CALL p_insert_department('Economía', 'Almería');
```

```
-- Insertar departamento duplicado:
-- CALL p_insert_department ('Economía', 'Almeria');
```

empleados/p_insert_employee.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: Procedimiento para insertar un nuevo Empleado, si la
-- startDate es null, poner la fecha actual del sistema
--

DELIMITER //
CREATE OR REPLACE PROCEDURE p_insert_employee (
    p_employee_id INT,
    p_department_id INT,
    p_boss_id INT,
    p_name_emp VARCHAR(64),
    p_start_date DATE,
    p_end_date DATE,
    p_salary DOUBLE,
    p_fee DOUBLE)
BEGIN
    IF (p_start_date IS NULL) THEN
        SET p_start_date = SYSDATE();
    END IF;
    INSERT INTO employees (employee_id, department_id, boss_id, name_emp,
salary, start_date, end_date, fee)
    VALUES (p_employee_id, p_department_id, p_boss_id, p_name_emp, p_salary,
p_start_date, p_end_date, p_fee);
END //
DELIMITER ;

-- CALL p_populate_db();
-- CALL p_insert_employee(6, 1, NULL, 'Daniel', '2020-09-15', NULL, 2500.0,
0.2);
```

Realice un procedimiento para igualar las comisiones de todos los empleados al valor de la comisión media:

empleados/p_equate_fees.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: Procedimiento para igualar las fees de todos los employees
-- a la media de las fees.
--
```

```

DELIMITER //
CREATE OR REPLACE PROCEDURE
  p_equate_fees()
BEGIN
  DECLARE avg_fee DOUBLE;
  SET avg_fee = (SELECT AVG(fee) FROM employees);
  -- Modifica TODOS los employees, no tiene WHERE
  UPDATE employees SET fee = avg_fee;
END //
DELIMITER ;

-- CALL p_populate();
-- CALL p_equate_fees();

```

Implemente un procedimiento que aplique un aumento a la comisión de un empleado en particular:

empleados/p_raise_fee.sql

```

--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: Procedimiento para aplicar un aumento dado a la comisión de
-- empleado concreto
--

DELIMITER //
CREATE OR REPLACE PROCEDURE
  p_raise_fee(id INT, amount DOUBLE)
BEGIN
  DECLARE e ROW TYPE OF employees;
  DECLARE new_fee DOUBLE;
  SELECT * INTO e -- el resultado del select lo almacena en la variable
    FROM employees
    WHERE employee_id = id;
  SET new_fee = e.fee + amount;
  UPDATE employees
    SET fee = new_fee
    WHERE employee_id = id;
END //
DELIMITER ;

-- CALL p_populate_db();
-- CALL p_raise_fee(4,0.3);

```

8.2. Funciones

Implemente una función que devuelva el número de empleados de una localidad concreta:

empleados/f_num_employees.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: Función que devuelve el numero de Employees de una
-- localidad
--

DELIMITER //
CREATE OR REPLACE FUNCTION
  f_num_employees(c VARCHAR(64)) RETURNS INT
BEGIN
  RETURN (
    SELECT COUNT(*)
    FROM employees e JOIN departments d
    ON (e.department_id = d.department_id)
    WHERE d.city = c
  );
END//
DELIMITER ;

-- SELECT *, f_num_employees(d.city) num_employees
-- FROM departments d
-- ;
```

Implemente una función que calcule la media de las comisiones de los empleados y use esa función dentro de un procedimiento almacenado para igualar las comisiones de todos los empleados al valor de la comisión media:

empleados/f_avg_fee.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: Ejemplo de uso de funciones dentro de procedimientos deben
-- almacenarse en variables
--

DELIMITER //
CREATE OR REPLACE FUNCTION
  f_avg_fee() RETURNS DOUBLE
BEGIN
  RETURN (
    SELECT AVG(fee)
    FROM employees
  );
END//
DELIMITER ;
```

```

);
END //
DELIMITER ;

-- El procedimiento puede usar la función
DELIMITER //
CREATE OR REPLACE PROCEDURE
  p_equate_fees()
BEGIN
  DECLARE af DOUBLE;
  SET af = f_avg_fee();
  UPDATE employees SET fee = af;
END//
DELIMITER ;

-- CALL p_populate_db();
-- CALL p_equate_fees();

```

8.3. Cursores

Utilice un cursor para recorrer todos los empleados y calcular el valor acumulado de los salarios:

empleados/f_sum_salaries.sql

```

--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: Ejemplo de función que recorre un cursor
--

DELIMITER //
CREATE OR REPLACE FUNCTION
  f_sum_salaries() RETURNS DECIMAL
BEGIN
  DECLARE total DECIMAL;
  DECLARE employee ROW TYPE OF employees;
  DECLARE done BOOLEAN DEFAULT FALSE;
  DECLARE cur_employees CURSOR FOR
    SELECT *
    FROM employees;
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET done := TRUE;
  SET total = 0;
  OPEN cur_employees;
  readLoop: LOOP
    FETCH cur_employees INTO employee;
    IF done THEN

```

```

        LEAVE readLoop;
    END IF;
    SET total = total + employee.salary;
END LOOP;
CLOSE cur_employees;
RETURN total;
END //
DELIMITER ;

```

8.4. Triggers (disparadores)

Implemente un disparador para evitar que un empleado sea su propio jefe:

empleados/t_self_boss.sql

```

--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: Comprueba que un empleado no puede ser su propio jefe.
--

DELIMITER //
CREATE OR REPLACE TRIGGER t_biu_self_boss
BEFORE INSERT OR UPDATE ON employees
FOR EACH ROW
BEGIN
    IF NEW.employee_id = NEW.boss_id THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RN-02: Un empleado no puede ser su propio jefe';
    END IF;
END //
DELIMITER ;

-- CALL p_populate_db();
-- CALL p_insert_employee(p_employee_id, p_department_id, p_boss_id,
p_name_emp, p_start_date, p_end_date, p_salary, p_fee)

```

Implemente un disparador que evite que modifique la comisión de un empleado en más de un 20%:

empleados/t_change_fee.sql

```

--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: La comisión no puede variar más de 0.2 (20 puntos)

```

```

porcentuales) por actualización.
-- Dentro del Trigger se tiene acceso a 'old', que almacena la tupla
-- con los valores antes de cambiar.

-- OPCIÓN 1: No se permite realizar el cambio en la comisión
DELIMITER //
CREATE OR REPLACE TRIGGER t_bu_change_fee
BEFORE UPDATE ON employees FOR EACH ROW
BEGIN
    IF OLD.fee IS NOT NULL AND NEW.fee IS NOT NULL
        AND ABS(NEW.fee - OLD.fee) > 0.2 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT =
            'RN-02: La diferencia absoluta de comisión no puede superar 0.2';
    END IF;
END //
DELIMITER ;

-- OPCIÓN 2: Se permite realizar el cambio al valor máximo permitido
/*
DELIMITER //
CREATE OR REPLACE TRIGGER t_bu_change_fee
BEFORE UPDATE ON employees FOR EACH ROW
BEGIN
    IF OLD.fee IS NOT NULL AND NEW.fee IS NOT NULL THEN
        IF NEW.fee - OLD.fee > 0.2 THEN
            SET NEW.fee = LEAST(OLD.fee + 0.2, 1);
        ELSEIF NEW.fee - OLD.fee < -0.2 THEN
            SET NEW.fee = GREATEST(OLD.fee - 0.2, 0);
        END IF;
    END IF;
END //
DELIMITER ;
*/

```

Implemente un disparador que evite que un departamento tenga más de cinco empleados:

empleados/t_max_employees_department.sql

```

--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: Ejemplo de Trigger para comprobar que un Departamento
-- no tiene más de 5 Empleados.

DELIMITER //
CREATE OR REPLACE TRIGGER t_biu_max_employees_department
BEFORE INSERT OR UPDATE ON employees
FOR EACH ROW
BEGIN

```

```

DECLARE v_employee_count INT DEFAULT 0;

IF NOT (NEW.department_id <=> OLD.department_id) THEN
    SELECT COUNT(*) INTO v_employee_count
    FROM employees
    WHERE department_id = NEW.department_id;

    IF v_employee_count > 4 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RN-01: Un departamento no puede tener más de
cinco empleados';
    END IF;
END IF;
END //
DELIMITER ;

-- CALL p_populate_db();
-- CALL p_insert_employee(6, 1, NULL, 'quinto empleado departamento 1',
null, NULL, 1500, 0);
-- CALL p_insert_employee(7, 1, NULL, 'sexto empleado departamento 1',
null, NULL, 1500, 0);

```

Implemente un disparador que en caso de insertar un empleado sin fecha de inicio, le ponga como fecha de inicio la fecha actual:

empleados/t_default_start_date.sql

```

-- Ejemplo de Trigger para almacenar en la startDate del contrato
-- de un empleado a la fecha actual del sistema en que caso de que la
-- startDate proporcionada sea NULL
DELIMITER //
CREATE OR REPLACE TRIGGER t_default_start_date
BEFORE INSERT ON employees FOR EACH ROW
BEGIN
    IF (new.start_date IS NULL) THEN
        SET new.start_date = SYSDATE();
    END IF;
END //
DELIMITER ;

```

Pruebas SQL

empleados/tests.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre 2024
-- Descripción: Tests negativos para la BD de Empleados
--
USE EmployeesDB;

-- =====
-- TABLA DE RESULTADOS
-- =====
CREATE OR REPLACE TABLE test_results (
    test_id VARCHAR(20) PRIMARY KEY,
    test_name VARCHAR(200) NOT NULL,
    test_message VARCHAR(500) NOT NULL,
    test_status ENUM('PASS', 'FAIL', 'ERROR') NOT NULL,
    execution_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- =====
-- PROCEDIMIENTO AUXILIAR
-- =====
DELIMITER //
CREATE OR REPLACE PROCEDURE p_log_test(
    IN p_test_id VARCHAR(20),
    IN p_message VARCHAR(500),
    IN p_status ENUM('PASS', 'FAIL', 'ERROR')
```

```

)
BEGIN
    INSERT INTO test_results(test_id, test_name, test_message, test_status)
    VALUES (p_test_id, SUBSTRING_INDEX(p_message, ':', 1), p_message,
p_status);
END //
DELIMITER ;

-- =====
-- TESTS (RN01 - RN09)
-- =====

DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_ri02_unique_employee_name()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RI02A', 'RI-02: El nombre del empleado debe ser
único', 'PASS');

    CALL p_populate();

    INSERT INTO employees (department_id, boss_id, name_emp, salary,
start_date, end_date, fee)
    VALUES (1, NULL, 'Pedro', 1500, '2024-01-01', NULL, 0.1);

    CALL p_log_test('RI02A', 'ERROR: Se permitió duplicar el nombre del
empleado', 'FAIL');
END //
DELIMITER ;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn01_department_name_city()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN01A', 'RN-01: La combinación de nombre y
localidad debe ser única', 'PASS');

    CALL p_populate();

    INSERT INTO departments (name_dep, city) VALUES ('Arte', 'Cádiz');

    CALL p_log_test('RN01A', 'ERROR: Se permitió repetir nombre y
localidad', 'FAIL');
END //
DELIMITER ;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_ri01_department_name_required()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION

```

```

        CALL p_log_test('RI01', 'RI-01: El nombre del departamento es
obligatorio', 'PASS');

    CALL p_populate();

    INSERT INTO departments (name_dep, city) VALUES (NULL, 'Madrid');

    CALL p_log_test('RI01', 'ERROR: Se permitió un departamento sin
nombre', 'FAIL');
END //
DELIMITER ;

DELIMITER //
DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_ck01_salary_positive()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('CK01', 'CK-01: salary debe ser > 0', 'PASS');

    CALL p_populate();

    INSERT INTO employees (department_id, boss_id, name_emp, salary,
start_date, end_date, fee)
        VALUES (2, NULL, 'Salario Negativo', -10, '2024-01-01', NULL, 0.1);

    CALL p_log_test('CK01', 'ERROR: Se permitió salary negativo', 'FAIL');
END //
DELIMITER ;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_ri02_salary_required()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RI02D', 'RI-02: El salario es obligatorio',
'PASS');

    CALL p_populate();

    INSERT INTO employees (department_id, boss_id, name_emp, salary,
start_date, end_date, fee)
        VALUES (1, NULL, 'Sin Salario', NULL, '2024-01-01', NULL, 0.1);

    CALL p_log_test('RI02D', 'ERROR: Se permitió un empleado sin salario',
'FAIL');
END //
DELIMITER ;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_ri02_fee_required()
BEGIN

```

```

DECLARE EXIT HANDLER FOR SQLEXCEPTION
    CALL p_log_test('RI02E', 'RI-02: La comisión es obligatoria',
'PASS');

CALL p_populate();

INSERT INTO employees (department_id, boss_id, name_emp, salary,
start_date, end_date, fee)
    VALUES (1, NULL, 'Sin Comisión', 1200, '2024-01-01', NULL, NULL);

CALL p_log_test('RI02E', 'ERROR: Se permitió un empleado sin comisión',
'FAIL');
END //
DELIMITER ;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn02_fee_range()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN02A', 'RN-02: fee debe estar entre 0 y 1',
'PASS');

    CALL p_populate();

    INSERT INTO employees (department_id, boss_id, name_emp, salary,
start_date, end_date, fee)
        VALUES (2, NULL, 'Fee Invalido', 1200, '2024-01-01', NULL, 1.5);

    CALL p_log_test('RN02A', 'ERROR: Se permitió fee fuera de rango',
'FAIL');
END //
DELIMITER ;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_ck02_dates_order()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('CK02', 'CK-02: start_date debe preceder a
end_date', 'PASS');

    CALL p_populate();

    INSERT INTO employees (department_id, boss_id, name_emp, salary,
start_date, end_date, fee)
        VALUES (2, NULL, 'Fecha Incorrecta', 1200, '2024-12-01',
'2024-01-01', 0.1);

    CALL p_log_test('CK02', 'ERROR: Se permitió start_date >= end_date',
'FAIL');
END //

```

```

DELIMITER ;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn02_self_boss()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN02B', 'RN-02: Un empleado no puede ser su propio jefe', 'PASS');

    CALL p_populate();

    INSERT INTO employees (employee_id, department_id, boss_id, name_emp, salary, start_date, end_date, fee)
        VALUES (20, 1, 20, 'JefeDeSiMismo', 1500, '2024-01-01', NULL, 0.1);

    CALL p_log_test('RN02B', 'ERROR: Se permitió boss_id = employee_id', 'FAIL');
END //
DELIMITER ;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn02_self_boss_update()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN02E', 'RN-02: UPDATE no puede asignar un empleado como su propio jefe', 'PASS');

    CALL p_populate();

    UPDATE employees SET boss_id = employee_id WHERE employee_id = 1;

    CALL p_log_test('RN02E', 'ERROR: UPDATE permitió que un empleado fuera su propio jefe', 'FAIL');
END //
DELIMITER ;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn01_max_department_insert()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN01B', 'RN-01: Máximo cinco empleados por departamento', 'PASS');

    CALL p_populate();

    INSERT INTO employees (department_id, boss_id, name_emp, salary, start_date, end_date, fee)
        VALUES (1, 1, 'Empleado6', 1200, '2024-02-01', NULL, 0.1);

    CALL p_log_test('RN01B', 'ERROR: Se permitió un sexto empleado en el

```

```

departamento', 'FAIL');
END //
DELIMITER ;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn01_max_department_transfer()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN01C', 'RN-01: No se puede trasladar a un
departamento lleno', 'PASS');

    CALL p_populate();

    UPDATE employees
    SET department_id = 1
    WHERE employee_id = 3;

    CALL p_log_test('RN01C', 'ERROR: Se permitió superar el máximo al
trasladar un empleado', 'FAIL');
END //
DELIMITER ;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn01_null_department_transfer()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN01D', 'RN-01: No se puede trasladar desde NULL a
un departamento lleno', 'PASS');

    CALL p_populate();

    UPDATE employees
    SET department_id = 1
    WHERE employee_id = 7;

    CALL p_log_test('RN01D', 'ERROR: Se permitió superar el máximo al
asignar desde NULL', 'FAIL');
END //
DELIMITER ;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn02_fee_delta()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN02C', 'RN-02: La comisión no puede aumentar más
de 0.2 puntos', 'PASS');

    CALL p_populate();

    UPDATE employees SET fee = 0.5 WHERE employee_id = 1;

```

```

    CALL p_log_test('RN02C', 'ERROR: Se permitió aumentar la comisión más
de 0.2 puntos', 'FAIL');
END //
DELIMITER ;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn02_fee_delta_decrease()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN02D', 'RN-02: La comisión no puede reducirse más
de 0.2 puntos', 'PASS');

    CALL p_populate();

    UPDATE employees SET fee = 0.2 WHERE employee_id = 2;

    CALL p_log_test('RN02D', 'ERROR: Se permitió reducir la comisión más de
0.2 puntos', 'FAIL');
END //
DELIMITER ;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_ri02_department_fk()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RI02C', 'RI-02: El departamento del empleado debe
existir', 'PASS');

    CALL p_populate();

    INSERT INTO employees (department_id, boss_id, name_emp, salary,
start_date, end_date, fee)
        VALUES (99, NULL, 'Dep Inexistente', 1200, '2024-01-01', NULL,
0.1);

    CALL p_log_test('RI02C', 'ERROR: Se permitió un department_id
inexistente', 'FAIL');
END //
DELIMITER ;

-- =====
-- ORQUESTADOR
-- =====

DELIMITER //
CREATE OR REPLACE PROCEDURE p_run_tests()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        BEGIN
            CALL p_log_test('POPULATE', 'ERROR: El populate falló. No se

```

```

ejecutaron los tests negativos.', 'ERROR');
    SELECT * FROM test_results ORDER BY execution_time, test_id;
    SELECT test_status, COUNT(*) AS total FROM test_results GROUP BY
test_status;
    END;

    DELETE FROM test_results;

    CALL p_populate();

    CALL p_test_ri02_unique_employee_name();
    CALL p_test_rn01_department_name_city();
    CALL p_test_ri01_department_name_required();
    CALL p_test_ck01_salary_positive();
    CALL p_test_ri02_salary_required();
    CALL p_test_ri02_fee_required();
    CALL p_test_rn02_fee_range();
    CALL p_test_ck02_dates_order();
    CALL p_test_rn02_self_boss();
    CALL p_test_rn02_self_boss_update();
    CALL p_test_rn01_max_department_insert();
    CALL p_test_rn01_max_department_transfer();
    CALL p_test_rn01_null_department_transfer();
    CALL p_test_rn02_fee_delta();
    CALL p_test_rn02_fee_delta_decrease();
    CALL p_test_ri02_department_fk();

    SELECT * FROM test_results ORDER BY execution_time, test_id;
    SELECT test_status, COUNT(*) AS total FROM test_results GROUP BY
test_status;
    END //
DELIMITER ;

CALL p_run_tests();

```

i Info

Versión PDF disponible